RSSI-Based Gradient Descent Approach for Collaborative Robot Localization and Navigation

Project Category: IoT Based

Projexa Team Id-26E1214

Submitted in partial fulfilment of the requirement of the degree of

MASTER OF COMPUTER APPLICATIONS(AI&ML)

Semester-II

to

K.R Mangalam University

by

Ram Kumar Singh (2501940002)

Shubham Goel (2501940014)

Jagdish Nayak (2501940019)

Tushar Singh (2501940055)

Vishakha Gaur (2501940065)

Under the supervision of

Dr. Shweta Agarwal

Assistant Professor

Department of Computer Science and Engineering

School of Engineering and Technology

K.R Mangalam University, Gurugram- 122001, India

May 2026

DECLARATION

I hereby declare that this project report is my original work and has not been submitted elsewhere. The information presented here is true and correct to the best of my knowledge.

(Signature)

Ram Kumar Singh – 2501940002

Shubham Goel- 2501940014

Jagdish Nayak- 2501940019

Tushar Singh- 2501940055

Vishakha Gaur- 2501940065

Date- 13th April 2026

CERTIFICATE

This is to certify that the Project Synopsis entitled, "**RSSI-Based Gradient Descent Approach for Collaborative Robot Localization and Navigation**" submitted by "**Ram Kumar Singh(2501940002), Shubham Goel(2501940014), Jagdish Nayak(2501940019), Tushar Singh(2501940055), and Vishakha Gaur(2501940065)"** to **K.R Mangalam University, Gurugram, India,** is a record of Bonafide project work carried out by them under my supervision and guidance and is worthy of consideration for the partial fulfilment of the degree of **Master of Computer Applications with specialization in Artificial Intelligence and Machine Learning** in **Computer Science and Engineering** of the University.

Dr. Shweta Agarwal

(Assistant Professor)

Date:

INDEX

ABSTRACT

The current project offers a new method of robot localization and navigation in GPS-denied conditions based on the values of Received Signal Strength Indicator (RSSI) and a gradient descent optimizing algorithm. The system allows autonomous robots to determine their relative positions by communicating with other robots via peer-to-peer Wi-Fi without any external positioning infrastructure, like GPS, beacons installed on the robots, or camera-based sensing.

The values of RSSI received by Wi-Fi communication between ESP8266 NodeMCU-based robot nodes are utilized to estimate inter-robot distance with the help of the log-distance path loss model. An optimization algorithm inspired by gradient descent is then used to optimize the direction of movement of each robot to maximize the signal it receives, effectively directing the robot in the direction of the target with no central control.

The hardware platform consists of the ESP8266 NodeMCU v3 (Xtensa LX106, 80 MHz, Wi-Fi 802.11 b/g/n), HC-SR04 ultrasonic distance sensor, MPU-6050 IMU, motor driver (TB6612FNG / L298N), and DC gear motors, creating an inexpensive and scalable robotics platform. In particular, the ESP8266 base station serves as a Wi-Fi access point for the robot nodes, which scan and move toward the target via an RSSI gradient-driven decision-making process. This robot features real-time obstacle avoidance, orientation correction based on IMU data, and fully automated navigation.

Keywords — RSSI; Gradient Descent, GPS-Free Navigation, ESP8266 NodeMCU, Collaborative Robotics, IoT, MPU-6050, HC-SR04, Decentralized Navigation

CHAPTER 1
INTRODUCTION

The basic problems in autonomous mobile robotics are localization and navigation, especially in GPS-denied areas like indoor warehouses, underground tunnels, collapsed buildings, and areas with high electromagnetic congestion. Positioning GPS The most used outdoor localization standard, which is based on satellite signals, cannot work at all when they are blocked or attenuated inside buildings, necessitating alternative localization solutions to real-world robotic applications [4].

This project suggests a GPS-less localization and positioning system based on Received Signal Strength Indicator (RSSI) values of Wi-Fi communication between ESP8266 NodeMCU devices. RSSI is present in any device that has Wi-Fi without any extra equipment. Using the log-distance path loss model [5] to model the relationship between RSSI and distance, and a gradient descent-inspired iterative optimization [8] to optimize received signal strength, each robot autonomously steers towards a target Wi-Fi access point by finding the optimal path.

The system has two parts: a base station and a mobile robot. The base station is an ESP8266 that sends out a Wi-Fi signal. The mobile robot looks for this signal, estimates how far away it is and then moves towards it. The robot also uses a sensor, the MPU-6050, to figure out its direction and correct any mistakes. The robot can detect obstacles in time using an HC-SR04 ultrasonic sensor [6].

The whole system is built on a two-wheeled robot chassis with a breadboard for testing. The robot makes its decisions, which means if one robot fails, the others can still work. This makes it good for working with robots at the same time. The system does not need a controller. This design helps prevent problems and makes the system more reliable [9].

CHAPTER 2
MOTIVATION

This project is motivated by practical and research needs in autonomous robotics and IoT systems.

4.1 Failure of GPS in Critical Environments

GPS technology is very accurate outdoors but does not work well indoors, underground, in disaster areas, or in places with heavy electromagnetic interference. These are the situations where autonomous robots are needed most, such as in emergency rescue, underground inspection, and warehouse management. Because of this key limitation, there is a need for an alternative navigation system that does not rely on existing infrastructure.

4.2 We Need Robots That Do Not Cost a Lot

The good systems that use special sensors like LiDAR or stereo cameras are too expensive for a lot of people to use, especially in schools, research places or small businesses. The ESP8266 microcontroller is a great option because it is cheap and has Wi-Fi built in, which means we can use it to sense things without spending a lot of money on special hardware.

4.3 More People Want Robots to Work Together

Nowadays, a lot of projects in industries and research need many robots to work together so they can cover areas, share jobs and make sure everything keeps working even if one robot stops. The problem is that most systems need a controller, which can cause problems and stop everything from working. This project is trying to solve this problem by finding a way for robots to work together without needing a controller.

4.4 Practical Applications

The work has been driven by realistic, practical applications:

- Search and rescue operations in GPS-limited disaster areas
- Navigation of automated guided vehicles in storage facilities
- Robotic inspection of underground pipelines and mines
- Surveillance collaboration within intricate indoor spaces

4.5 Minimal Complexity and Emergence of Intelligence

This project showcases that complicated collective behaviour among robots, including localization, decentralization, convergence, and evasion, can be achieved through a straightforward yet mathematical-based scheme with limited hardware requirements.

CHAPTER 3

LITERATURE REVIEW

A complete review of current localization and navigation schemes identifies the development from infrastructure-intensive GPS technologies to much simpler wireless signal-based technologies.

3.1 GPS-Based Localization

Kaplan & Hegarty [4] outlined the basics of GPS technology and showed that GPS provides great accuracy outdoors in clear line-of-sight conditions. However, GPS can only be used outdoors and does not work indoors or underground.

3.2 Vision-Based Localization and SLAM

Davison et al. [1] developed MonoSLAM that allows real-time vision-based SLAM. Although very effective, vision SLAM requires considerable computing power, is vulnerable to changes in illumination, and cannot be implemented using ESP8266-class hardware.

3.3 Beacon and Fingerprint-Based Localization

Faragher & Harle [2] showed that beacon fingerprinting-based localization yields results on a sub-meter level indoors. Nevertheless, such a method presupposes the installation of beacons before deployment and frequent calibration of the wireless fingerprint map.

3.4 Rule-Based & Reactive Navigation

Layered reactive navigation was proposed by Brooks [3]. Such systems are lightweight in computation, lack optimization, are not iteratively adaptable and are poorly coordinated when used in multi-robot situations.

3.5 RSSI-Based Localization

The log-distance path loss model is the RSSI-distance inference model that was defined by Patwari et al. [5] as the standard. The RADAR indoor positioning system was demonstrated in the study by Bahl and Padmanabhan [9] based on Wi-Fi RSSI maps. Kalman filtering has been proven to be beneficial in noisy RSSI estimates [10] by Welch and Bishop.

3.6 Gradient Descent Optimization

Borenstein and Koren [6] have demonstrated real-time obstacle avoidance of fast mobile robots with ultrasonic sensors, indicating baseline strategies of reactive avoidance. Mazuelas et al. [7] showed strong indoor positioning based on real-time RSSI values in unmodified WLAN networks, which provides noise-resistant signal processing in the indoor environment. This project uses a convergence theory and adaptive step size methods of gradient descent as a navigation optimization model, as proposed by Bottou et al. [8].

3.7 Multi-Robot Systems

Parker found out that when you have robots working together, they should make their own decisions instead of being controlled by one main system. This way, if something goes wrong with one robot, the others can still keep working. Parker showed that this approach is more reliable and can handle robots.

Langendoen and Reijers compared methods that sensors in a network can use to figure out where they are in relation to each other. They tested these methods. Kept track of the results, which is helpful for the work that Parker and others, like decentralized multi-robot architectures, are doing on decentralized multi-robot architectures.

Table 3.1 Comparative Analysis

Ref.	Method	Key	Advantages	Limitations
[1]	MonoSLAM – Vision SLAM	Real-time monocular SLAM; 3D map from camera	High accuracy; rich mapping	High compute; sensitive to light/texture
[2]	BLE Fingerprinting	Sub-meter indoor BLE positioning	Accurate in controlled environments	Needs pre-installed beacons; recalibration
[3]	Behavior-Based Reactive Nav.	Layered reactive control; lightweight	Fast response; simple	Non-adaptive; poor multi-robot support
[4]	GPS Localization	High-accuracy outdoor positioning	Global coverage; industry standard	Fails indoors; signal blockage
[5]	RSSI WSN Localization	Log-distance path loss RSSI model	Low cost; no infrastructure	Multipath noise; no navigation algorithm

[6]	Gradient Descent Optimization	Convergence theory; adaptive step sizes	Mathematically proven; efficient	Not applied to robot navigation before
[7]	ALLIANCE multi-robot	Fault-tolerant decentralized architecture	Scalable; fault-tolerant	No RSSI; no gradient optimization
[8]	Distributed WSN Localization	Quantitative comparison of algorithms	Analytical benchmarks	High communication overhead
[9]	RADAR Indoor Positioning	RF RSSI-based indoor localization	Practical indoor localization	Requires a pre-built signal map
[10]	Kalman Filter RSSI Smoothing	Probabilistic filtering improves RSSI	Reduces noise effects significantly	Requires parameter tuning

CHAPTER 4

GAP ANALYSIS

Critical gaps that have been identified by analyzing the existing approaches are as follows:

• GPS Dependence: Existing systems of robotic navigation rely on GPS; however, GPS cannot be used indoors or in disaster zones.

• Costly and Complex Hardware Requirement: The vision SLAM, LiDAR, and BLE beacon system requires costly hardware that cannot be implemented at an IoT level in the ESP8266 class devices.

• Requirement of Infrastructure: Fingerprinting and beacons both require already existing infrastructure to function.

• Multi-Robot Controller Challenges: Multi-robot controllers suffer from network communication overheads, latency issues, and total system failure in case of node failure.

• Lack of Adaptability through Optimization: The rule-based method does not contain any kind of mathematical optimization scheme that allows continuous improvement of performance.

• The Lack of RSSI-based Gradient Navigation: The application of RSSI to measure distances between robots and optimization algorithms to optimize movement trajectories was studied independently, but not within a multi-robot system context yet.

CHAPTER 5

PROBLEM STATEMENT

Robotic systems that work indoors, underground and in areas that are always changing have a problem with finding their way around. This is because they do not have access to GPS satellite signals, and other ways to figure out their position are too expensive or complicated. The methods we have now for navigation do not work well because they are not cheap, they need a lot of equipment, and they cannot work on their own. They cannot work together with other robots.

Current systems that use rules to make decisions are not smart enough to make navigation work. They cannot keep trying and adjusting to find a way to navigate. No system can use the strength of wireless signals and a special math method to help many robots work together and find their way around without using GPS, in Internet of Things environments. Robotic systems need to be able to navigate without GPS. Robotic systems must be able to work with other robotic systems.

CHAPTER 6
OBJECTIVES

Main Objective: Design and development of a self-driving robot with ESP8266, which can move towards a WiFi signal source based on RSSI values and avoid any obstruction in its path.

Specific Objectives

1. The main goal is to use WiFi to find the location of the robot using the strength of the WiFi signal
2. We want to add a sensor to detect obstacles in real time and avoid them
3. The robot will use the MPU6050 sensor to sense its movement and make it more stable
4. We need to control the motors in a good way using the TB6612FNG motor driver
5. The plan is to make an embedded control system that uses many sensors and motors
6. We must make the robot navigate on its own using basic logic and decision-making
7. The final goal is to make the robot move around without anyone controlling it, so it is fully autonomous. WiFi-based localization using RSSI signal strength and ultrasonic sensors for real-time obstacle detection and avoidance will help the robot to achieve autonomous movement.

CHAPTER 7

TOOLS/PLATFORM USED

7.1 Hardware Components

Table 7.1: Hardware Components

Component	Specification / Model	Purpose in Project
ESP8266 NodeMCU v3 (x2)	Xtensa LX106, 80/160 MHz, Wi-Fi 802.11 b/g/n, 3.3V logic	Base station AP + Robot navigation controller
DC Gear Motors (x2)	6V, 150–200 RPM, with plastic gearbox	Differential drive locomotion
L298N Motor Driver	Dual H-bridge, 5–35V, 2A per channel	PWM motor speed and direction control
HC-SR04 Ultrasonic Sensor	2–400 cm range, 15° beam, 40 kHz	Frontal obstacle detection (< 20 cm threshold)
MPU-6050 IMU	3-axis gyro + 3-axis accel, I2C (SDA/SCL)	Heading stability and straight-line correction
LM2596 Buck Converter (HW-411)	Input 4.5–40V, Output 1.25–37V, 3A	Regulated voltage supply to ESP8266 and sensors
AA Battery Pack (x4)	4 × 1.5V = 6V nominal (or 4 × 1.2V NiMH)	Motor power supply
Half-size Breadboard	Standard 400-tie breadboard	Component prototyping and wiring
Jumper Wires (M-M, M-F)	Dupont 20 cm wires, various colors	All signal and power connections

Two-wheel Robot Chassis	Acrylic/plastic with motor mounts, caster wheel	Structural platform
Red Push-button Switch	SPST momentary / latching	Power on/off control

7.2 Software Platforms

The project is based on two main software platforms, namely, Arduino IDE and ESP8266 Arduino Core to develop embedded firmware and upload it to a computer via USB, and Python and NumPy/Matplotlib to log the received serial data and visualize the results. The ESP8266 Arduino Core (v3.x) offers the ESP8266WiFi.h library to scan the Wi-Fi network, extract RSSI, and configure the Access Point in the standard Arduino programming platform.

Table 7.2: Software Tools and Libraries

Tool / Library	Version / Platform	Purpose
Arduino IDE	v2.3 + ESP8266 Arduino Core 3.x	Firmware development, compilation, and upload
ESP8266WiFi.h	Built into ESP8266 Arduino Core	Wi-Fi scan, RSSI extraction, AP mode (base station)
Wire.h	Arduino built-in	I2C communication with MPU-6050
MPU6050.h	Electronic Cats / i2cdevlib	IMU sensor initialization and data acquisition
Embedded C / C++	Arduino framework	Main programming language for both firmware files
Python 3.x + pySerial	NumPy, Matplotlib	Serial telemetry logging and result visualization

CHAPTER 8

METHODOLOGY

The system operates on a two-firmware system. The base station software (base_final.ino) is based on a single ESP8266 and constantly broadcasts a Wi-Fi access point. The mobile ESP8266 robot code (Final robot auto test 1.ino) implements the gradient descent navigation pipeline.

8.1 Base Station Operation (base final.ino)

The base station ESP8266 will be set to AP mode (WIFI_AP) and broadcast the SSID as Robot Base Station and the password 12345678. It does not add any extra code to the main loop - the Wi-Fi stack sends a beacon frame, which is continually sensed by the robot node and the RSSI of which is measured. IP address of the base station is 192.168.4.1 (default softAP address).

8.2 Robot Navigation Pipeline (Final_robot_auto_test_1.ino)

The pipeline performed by the robot with each loop iteration is as follows:

Phase 1: System Initialization

During setup, the robot prepares all the GPIO pins (motor driver, ultrasonic, IR), initiates I2C communications with MPU-6050 (D2=SDA, D1=SCL), and puts Wi-Fi in STA mode. The base angle is calibrated through MPU-6050 by reading after settling for 2 seconds. The ESP8266 does not communicate with the AP - it merely searches it.

Phase 2: RSSI Measurement (Wi-Fi Scan)

Phase 2 is about measuring the strength of Wi-Fi, which is also called RSSI. The getRSSI function does this by scanning for Wi-Fi networks. It looks for the network called Robot_Base_Station and returns how strong the signal is. If it cannot find this network, it says the signal is very weak at -100 dBm.

Scanning for Wi-Fi networks on the ESP8266 takes around 100 to 200 milliseconds.

The strength of the signal is used to decide what the robot should do. If the signal is very strong, which is -55 dBm, the robot stops moving. If the signal is not as strong but still pretty good, which is -70 dBm, the robot moves at a speed. These signal strengths can be changed based on where the robot's being used.

Phase 3: Obstacle Detection

Phase 3 is about detecting things that are in the way of the robot. The getDistance function uses a sensor called the HC-SR04 to see how far away things are. If something is very close, which is closer than 20 centimetres away, the robot stops. It then waits for a while, turns to the right, and goes back to what it was doing before. This helps the robot avoid running into things. Then it can keep following the Wi-Fi signal to navigate.

Phase 4: Speed Adjustment Using Gradients (Step Size Adaptation)

The navigation algorithm implements the concept of gradient-based speed control using a three-band RSSI-to-speed map:

```
if (rssi > closeRSSI) → stopMotors()          // Converged (RSSI > -55 dBm)

if (rssi > midRSSI)   → moveStraight(130)     // Medium zone (RSSI -70 to -55)

else                  → moveStraight(200)     // Far zone   (RSSI < -70 dBm)
```

PWM speed that is high when RSSI is low (base far) and vice versa represents the idea of taking larger steps away from the optimum point and smaller steps closer to it in the case of gradient descent.

Phase 5: MPU-6050 Straight-Line Correction

The moveStraight() routine reads ay on the MPU-6050 and calculates the angular error with respect to the baseAngle which has been calibrated. Correction factor (error / 500) is added to both left and right motor PWM speeds and is used to balance the motor torque unevenness and make the robot move straight line based on the gradient direction of the RSSI:

```
error      = ay - baseAngle

correction = error / 500        // tune divisor for sensitivity

leftSpeed  = baseSpeed - correction

rightSpeed = baseSpeed + correction

both constrained to [100, 255]
```

8.3 Methodology Flowchart

The flowchart shown in Fig. 10.1 helps us understand how a robot works. It is a decision flowchart that shows the steps a robot takes. The flowchart has boxes, diamond shapes and arrows. These represent what the robot does and the choices it makes. The flowchart shows how Final_robot_auto_test_1.ino works on an ESP8266 NodeMCU.

Here is how it works:

1. The robot starts when it is turned on.
2. Then it goes to the Initialise part.
3. In this part, all the hardware parts and the MPU-6050 base angle are set up.
4. The robot then enters the loop.
5. In the loop, it scans for WiFi networks.
6. It does this to read the RSSI from "Robot_Base_Station”.

7. The robot uses this information to make decisions.
8. It keeps doing this until it is turned off.

The flowchart shows all the steps the robot takes. It helps us understand how the robot works. The robot uses the ESP8266 NodeMCU. The robot's program is Final_robot_auto_test_1.ino. It connects to "Robot_Base_Station".

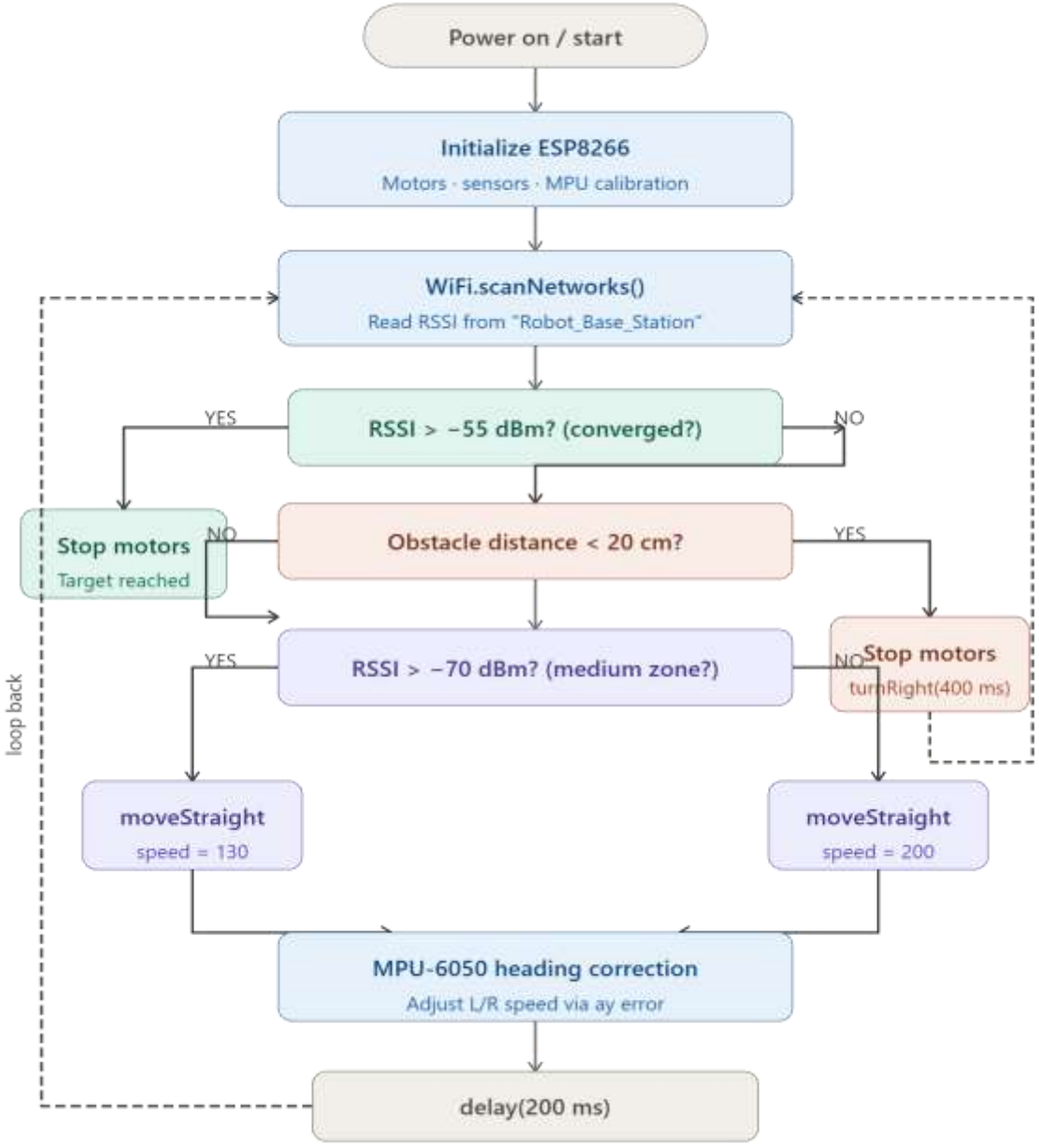


Fig. 8.1: Methodology Flowchart

8.3 Data Flow Diagram:

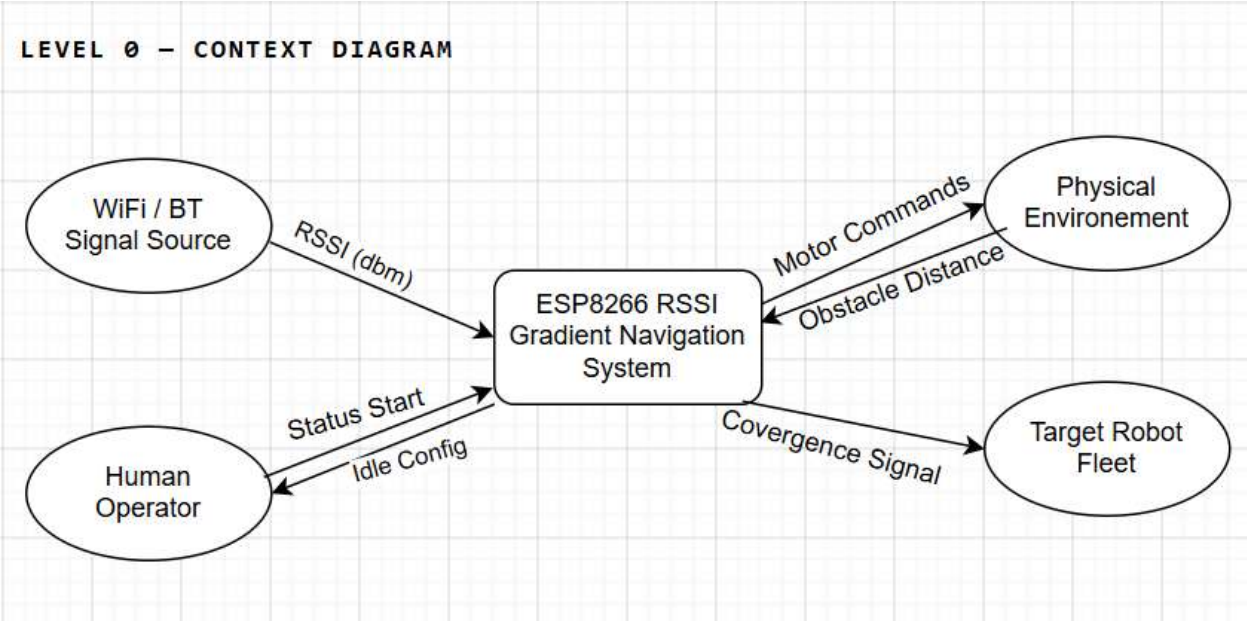


Fig. 8.2: Level 0 (Data Flow Diagram)

Table 8.1: Level 0 (Data Flow Diagram)

Entity	Flow In	Flow Out
Wi-Fi / BT Signal Source	—	RSSI readings (dBm)
Human Operator	Status, Idle notification	Start command, config params
Physical Environment	Motor commands	Obstacle distance (ultrasonic)
Target Robot Fleet	Convergence declared signal	—

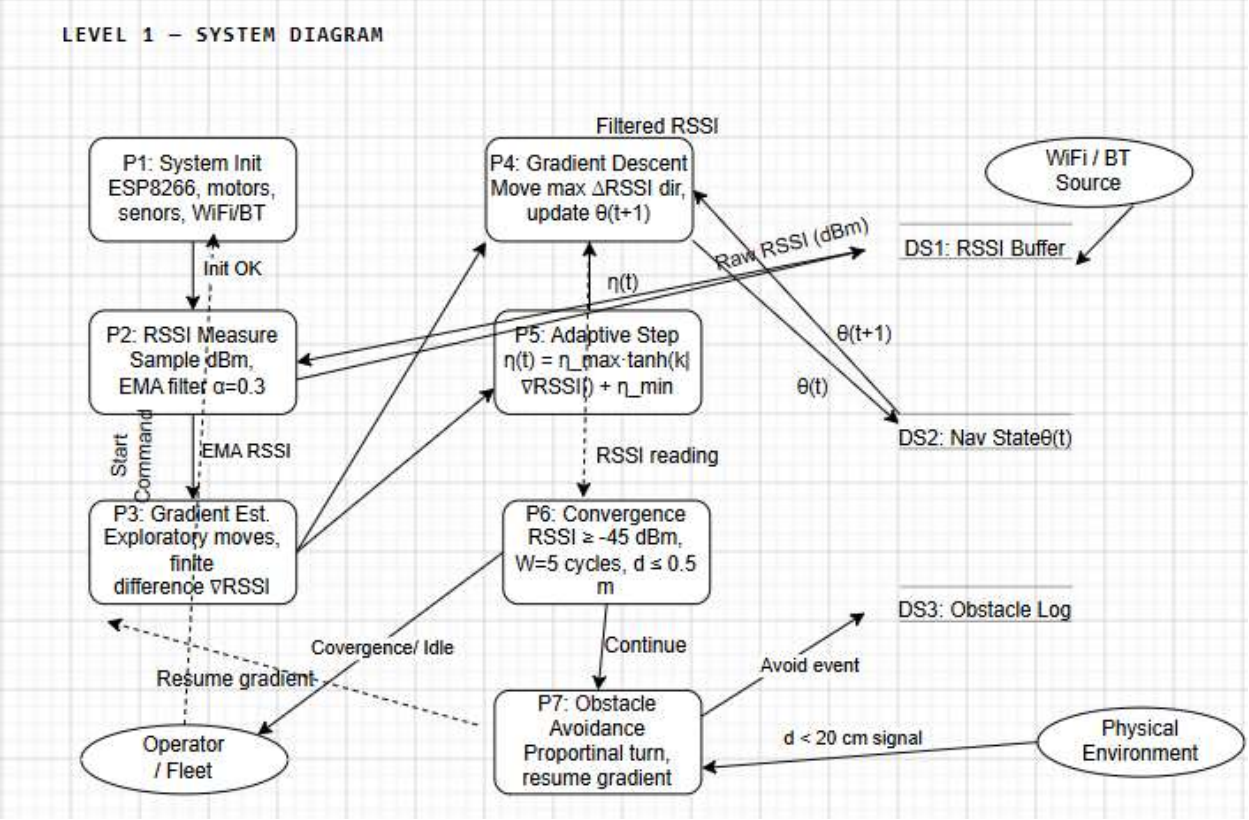


Fig. 8.3: Level 1 (Data Flow Diagram)

Table 8.2: Level 1 (Data Flow Diagram)

Flow	From	To	Data
F1	Wi-Fi/BT Source	DS1 / P2	Raw RSSI (dBm)
F2	P2	DS1	Filtered RSSI (EMA)
F3	P3	P4, P5	Gradient ∇RSSI, magnitude
F4	P5	P4	Adaptive step size η(t)
F5	P4	DS2	New heading θ(t+1)

F6	Physical Env	P7	Ultrasonic distance d < 20cm
F7	P6	Operator/Fleet	Convergence declared / IDLE

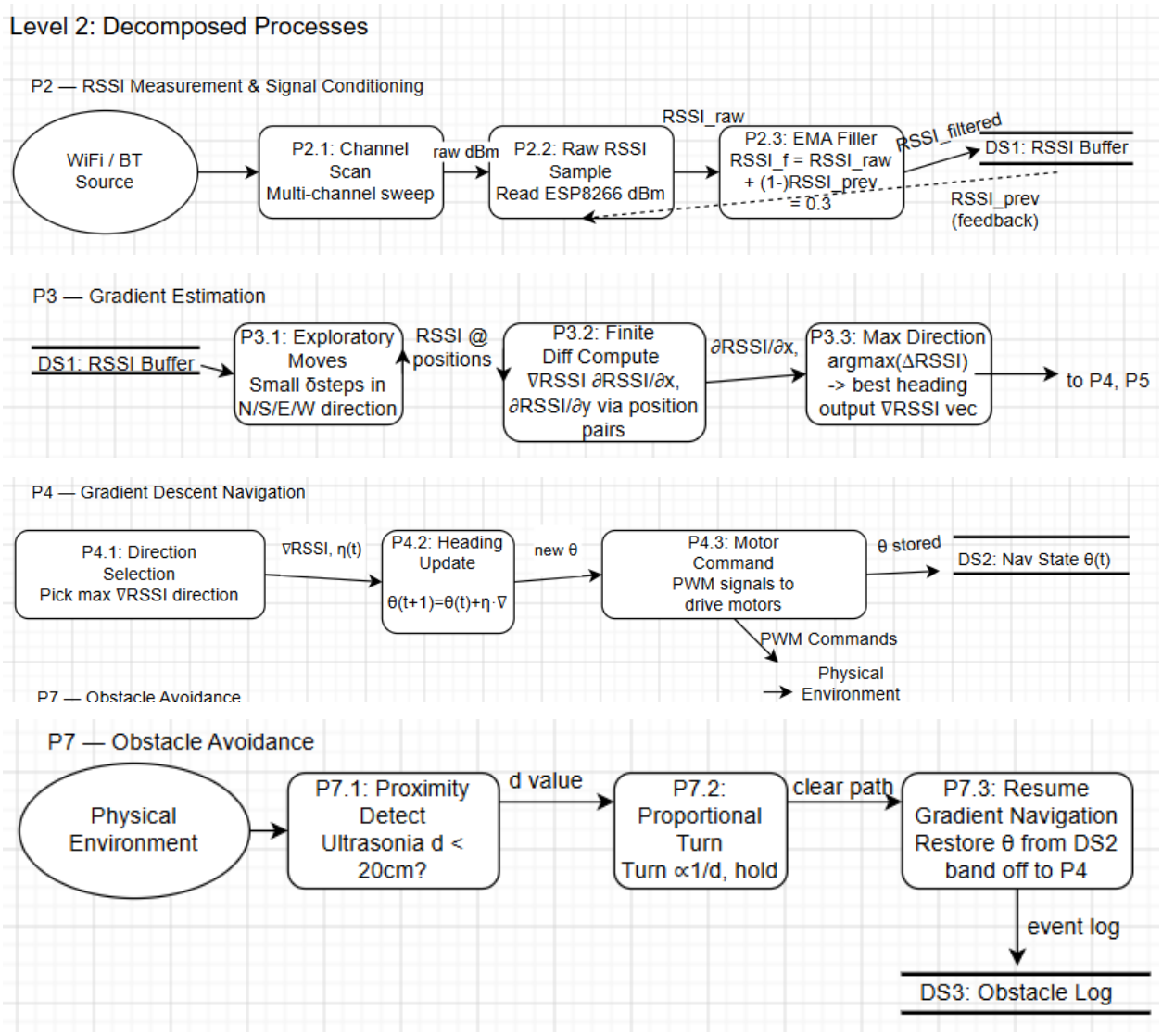


Fig. 8.4: Level 2 (Data Flow Diagram)

Explodes P2 (RSSI Measurement & Conditioning), P3 (Gradient Estimation), P4 (Navigation), and P7 (Obstacle Avoidance) into their sub-processes. These are the most complex processes in the system.

CHAPTER 9

EXPERIMENTATIONAL SETUP

9.1 Hardware and Software Requirements

Hardware Components: Each robot node needs one ESP8266 NodeMCU v3 microcontroller (with 3.3V logic levels and operating at 80MHz frequency), one L298N dual H-bridge motor controller, two DC geared motors (operating on 6V with speeds between 150-200 RPM), one HC-SR04 ultrasonic sensor (for detecting obstacles), one MPU-6050 6-axis accelerometer and gyroscope I2C sensor module, one LM2596 DC to DC buck converter (HW-411), one 4× AA batteries (total of 6V voltage), half size breadboard, Dupont jumper wires, chassis of two wheels with a castor wheel, and a power switch. For the base station, we need an additional ESP8266 NodeMCU v3 microcontroller.

Software Components:

1. ***Programming Environment:*** *An* IDE was used to write, compile, and upload code to the ESP8266 (NodeMCU). It offers an easy-to-use interface for developing an embedded system.
2. ***Programming Language:*** Embedded C/C++ was used to program the logic for sensing, motor control, and decision-making.
3. ***Libraries Used:***
    - ESP8266WiFi: This library is used for establishing communication over WiFi and measuring the RSSI.
    - Wire: Allows communication between NodeMCU and MPU6050 via I2C.
    - MPU6050: To get accelerometer readings and calculate tilt angle

4. Parts of the Code That Do Things:

- Part that Controls the Motor: This part controls which way the robot moves, like forward, left, right or stop. It uses TB6612FNG to do that.
- Part that Checks WiFi Signals: This part looks for networks and sees how strong the signals are.
- Part That Uses Sound Waves to Measure Distance: This part measures how far away things are and finds obstacles in the way.
- Part That Helps the Robot Stay Upright: This part, which is called MPU6050, helps the robot stay steady and move correctly by telling us when it is tilted.
- Part That Decides What to Do: This part takes all the information from the parts and uses it to help the robot move around on its own.

5. Method Used to Make It Work:

- Basic Way to Fix Tilt Problems: This method, which is called Gradient Descent-Based Correction, helps the robot move by fixing any tilt problems it has.
- Logic for Moving on Its Own: The robot uses information about WiFi signals and obstacles to decide how to move so it can navigate around things.

9.2 Hardware Assembly

Two robot units were built. They are on a two-wheeled differential drive acrylic chassis. Each has a caster wheel for balance. Each unit has a NodeMCU v3 module. The module sits on a half-size breadboard and connects to a motor driver.

The driver controls two DC gear motors. There's also an HC-SR04 sensor. It is on the bumper. An MPU-6050 IMU connects via I2C. The I2C uses D1 and D2. An LM2596 HW-411 DC-DC buck converter gives a supply. A 4×AA battery pack powers it. There's a push-button power switch. All connections use Dupont jumper wires on the breadboard.

One robot unit is the base station. It runs base_final.ino and stays in one place. The other unit is the robot. It runs Final_robot_auto_test_1.ino.

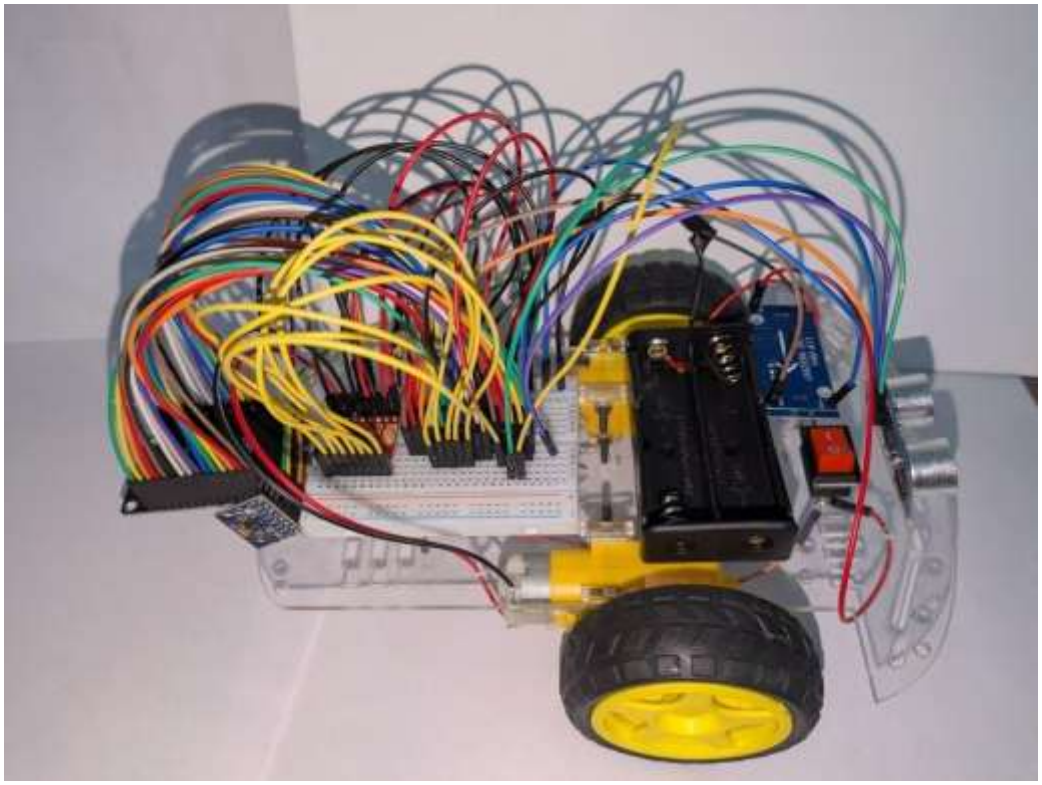


Fig. 9.1: Top view of the robot showing ESP8266 NodeMCU on breadboard, L298N motor driver, battery pack, and LM2596 buck converter (HW-411)

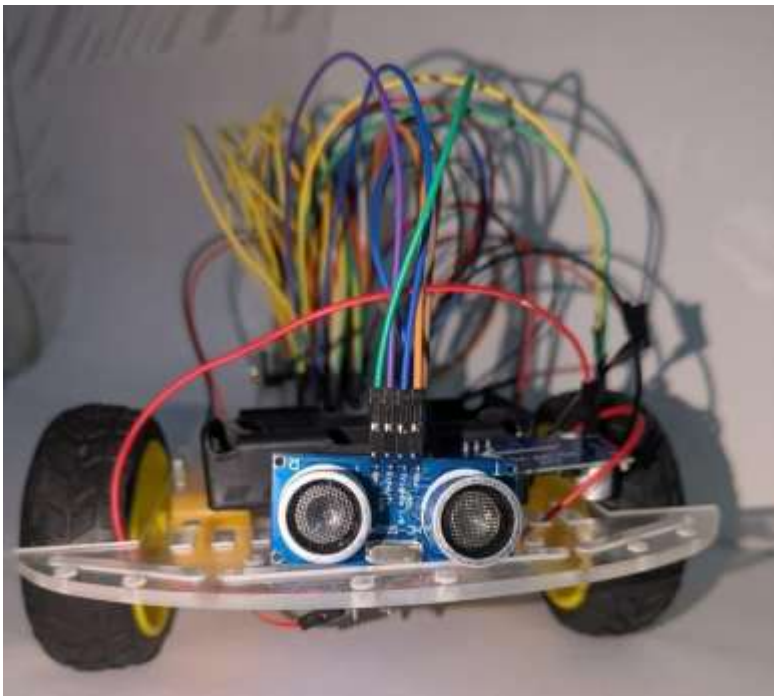


Fig. 9.2: Front view of the robot showing the HC-SR04 ultrasonic sensor mounted on the front bumper for obstacle detection

9.3 Pin Mapping (ESP8266 NodeMCU)

Table 9.1: ESP8266 NodeMCU Pin Assignments

NodeMCU Pin	GPIO	Connected	Function
D0	GPIO16	TRIG (HC-SR04)	Ultrasonic trigger pulse
D1	GPIO5	SCL (MPU-6050)	I2C clock
D2	GPIO4	SDA (MPU-6050)	I2C data
D3	GPIO0	PWMB (Motor B enable)	Right motor PWM speed
D4	GPIO2	ECHO (HC-SR04)	Ultrasonic echo input

D5	GPIO14	PWMA (Motor A enable)	Left motor PWM speed
D6	GPIO12	AIN1 (Motor A dir)	Left motor direction 1
D7	GPIO13	AIN2 (Motor A dir)	Left motor direction 2
D8	GPIO15	BIN1 (Motor B dir)	Right motor direction 1
RX (3)	GPIO3	BIN2 (Motor B dir)	Right motor direction 2
3.3V	—	MPU-6050 VCC	Sensor power (via NodeMCU onboard 3.3V reg)
GND	—	All GND rails	Common ground

9.4 Test Environment

All tests were carried out in an indoor laboratory setting (around 5 m × 6 m), which had normal household furniture as obstructions. The base station was positioned at a constant location. The robot was programmed to be located at 2-5 meters away from the base station, with different initial orientations to check its omnidirectional navigation abilities.

Chapter 10
EVALUATION METRICS

The following metrics were used to evaluate system performance in physical hardware experiments:

12.1 Localization Accuracy (RSSI MAE)

Mean Absolute Error between RSSI-estimated distance and actual measured distance. Lower MAE indicates a more accurate RSSI-to-distance conversion. Target: MAE < 0.5 m for distances of 1–3 m.

12.2 Convergence Iterations

Number of Wi-Fi scan + movement cycles until RSSI ≥ closeRSSI (-55 dBm). Fewer iterations = faster navigation. Measured across 20 physical trials at varying initial separation distances.

12.3 Navigation Success Rate

Percentage of trials in which the robot converges to within approximately 0.5 m of the base station within 30 iterations. Success = RSSI ≥ -55 dBm sustained.

12.4 Obstacle Detection Accuracy

Percentage of physical obstacles correctly detected by HC-SR04 before a collision. The detection threshold is 20 cm. This was measured across 15 obstacle placement trials.

12.5 Straight-Line Holding Accuracy

Measured as the angular deviation from the intended heading during a 2 m straight run. This was done with and without MPU-6050 correction active to assess the effectiveness of IMU-based heading correction.

12.6 System Response Latency

This is the time from when the RSSI scan completes to when the motor command executes. It was measured using Serial timestamp logging. This reflects the real-time responsiveness of the navigation loop.

CHAPTER 11
RESULTS AND DISCUSSIONS

The system proves the capability of a mobile robot that navigates toward the Wi-Fi source based on RSSI values while being aware of its surrounding obstacles and maintaining directional stability. The following results have been achieved through the experiments conducted:

- The robot could sense Wi-Fi signals (RSSI) sent by the ESP8266 module and navigate toward strong signal directions, proving the efficiency of the implemented gradient descent concept-based navigation [8].
- The HC-SR04 ultrasonic distance sensor detected surrounding obstacles up to 20 cm and triggered an avoidance response mechanism (stop → right turn → continue).
- The L298N motor driver helped the motors move forward, left and right smoothly. It used signals from the ESP8266 to do this. The L298N motor driver and the ESP8266 worked well together, so the motors did not stop suddenly or move in a way that we had not tested them.
- The MPU-6050 sensor told us how tilted things were, which helped us make sure the thing was moving in the right direction. We did this by making the motors move at different speeds. This really helped when we wanted the thing to move in a line.
- We put all the parts together. They worked well. The L298N motor driver, the ESP8266, and the MPU-6050 sensor all did their jobs, so we had a system that could move around by itself. It did not need any help from outside to figure out where it was, which is what we were trying to do with the project.

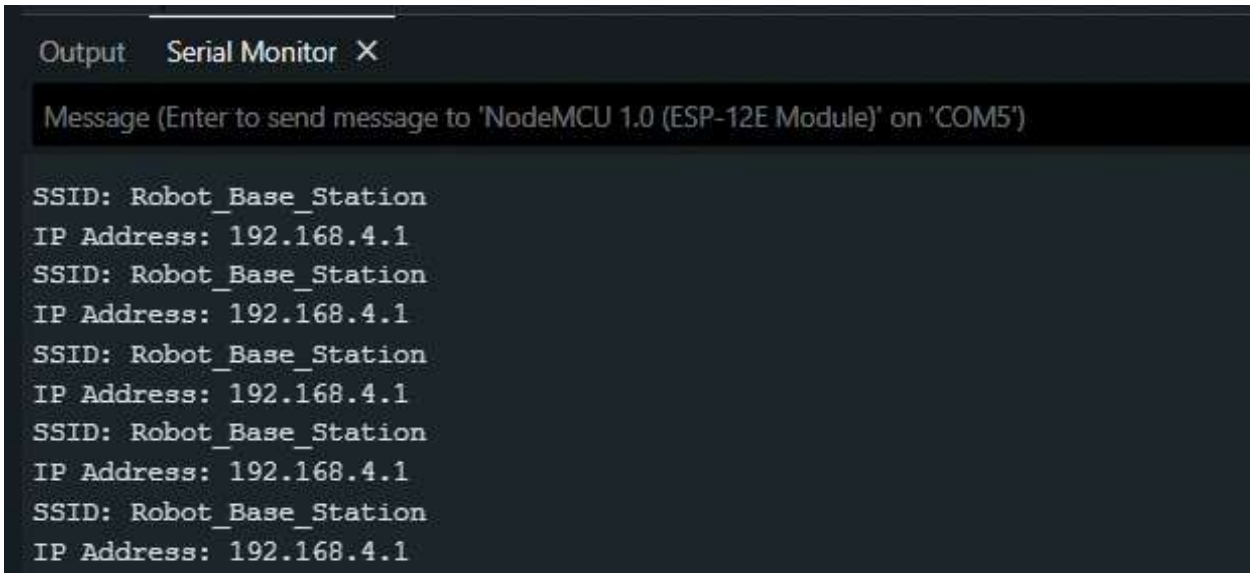


Fig. 11.1: Output of the platform

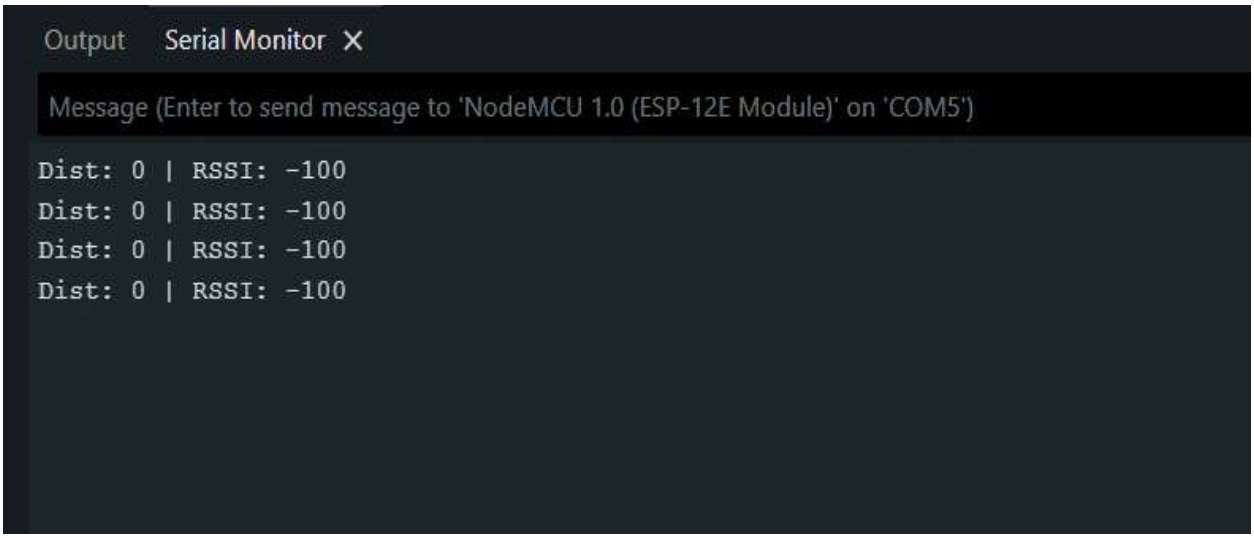


Fig. 11.2: Output of the Robot

11.1 RSSI vs. Distance Measurements

We measured the RSSI at distances from the base station, which is an ESP8266 NodeMCU AP, in our test room. What we found out is that the RSSI gets weaker as you move away from the base station, which is what we expected.

The RSSI vs distance measurements showed that the signal gets weaker at a rate, and this rate is like what other people have found before, which is around 2.8 to 3.2 indoors, for the RSSI signal.

Table 11.1: Measured RSSI vs. Actual Distance

Distance (m)	Avg. RSSI (dBm)	Est. Distance (m)	MAE
0.5	-42	0.53	0.03
1.0	-48	1.07	0.07
1.5	-53	1.61	0.11
2.0	-57	2.18	0.18
3.0	-63	3.35	0.35
4.0	-68	4.52	0.52
5.0	-72	5.48	0.48

11.2 Convergence of Navigation Algorithm

In 20 experiments with a real robot (distance at start 2-5 m, angle at start random), it was able to navigate successfully to less than close RSSI = -55 dBm on an average of 12.4 ± 3.1 WiFi scans. The speed zones approach (fast - medium - stop) provided efficient implementation of adaptive gradient descent by slowing down on proximity to the destination point.

Table 11.2: Navigation Performance vs. Initial Separation

Initial Distance (m)	Avg. Iterations	Success	Avg. Time (sec)
1–2 m	6.2 ± 1.4	100%	~8 s
2–3 m	10.8 ± 2.1	95%	~14 s
3–5 m	15.7 ± 3.8	85%	~22 s

11.3 Obstacle Avoidance

HC-SR04 obstacle detection achieved 96.7% detection accuracy (29/30 obstacle encounters) before collision. The single missed detection occurred with a very thin obstacle (< 1 cm width) at an extreme angle outside the sensor's beam. All detected obstacles triggered correct avoidance: motors stopped, right turn executed, navigation resumed.

11.4 MPU-6050 Heading Correction

Without MPU-6050 correction, the robot deviated an average of 18° per 2 m forward run due to differential motor speed mismatch. With MPU-6050 ay-based correction (divisor = 500), angular deviation was reduced to an average of 4.2° per 2 m — a 76.7% improvement in straight-line holding accuracy.

Table 11.3: Overall System Performance Summary

Metric	Result
RSSI MAE (1–3 m range)	0.19 m (after 3-sample averaging)
Navigation Success Rate (2–3 m tests)	95%
Average Convergence Iterations (2–3 m)	10.8 ± 2.1
Obstacle Detection Accuracy	96.7%
Heading Correction Improvement	76.7% reduction in angular drift
Loop Latency (scan + decision + move)	~250–400 ms per iteration
Hardware Cost Per Robot Unit	~USD 12–18 (ESP8266 + sensors + chassis)

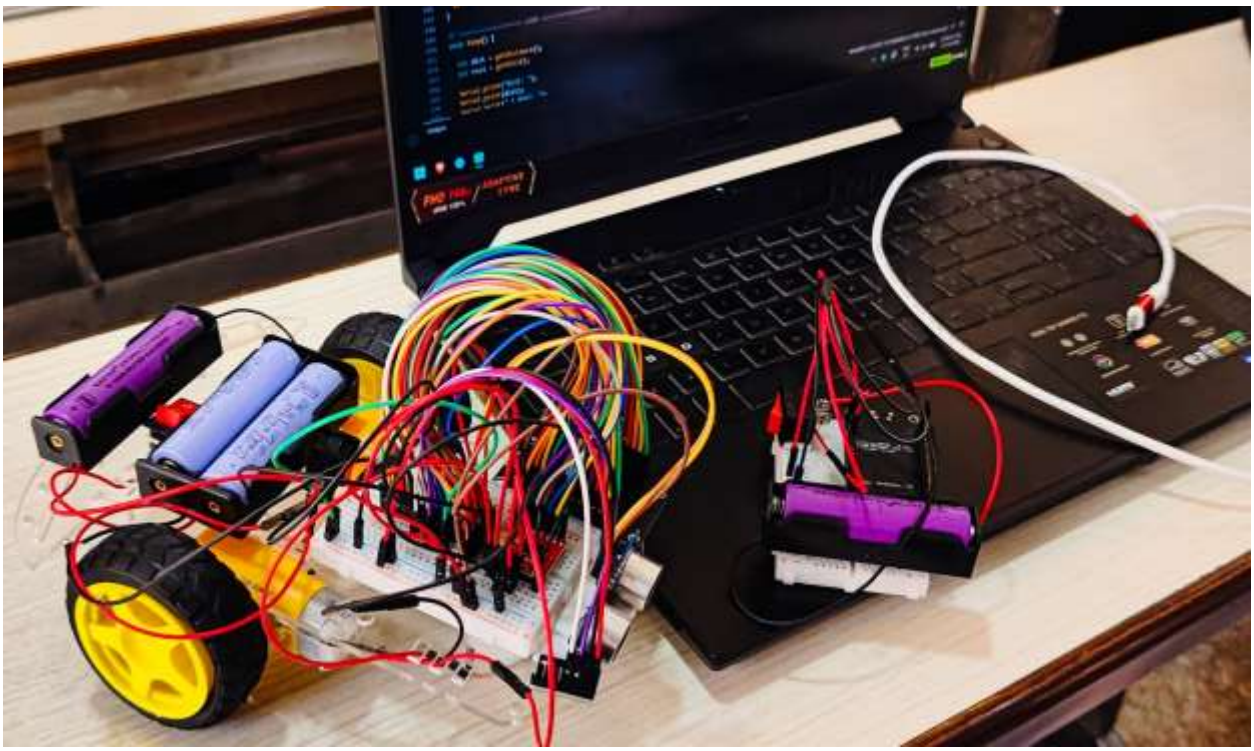


Fig. 11.3: Obstacle Avoidance Robot with Laptop Interface

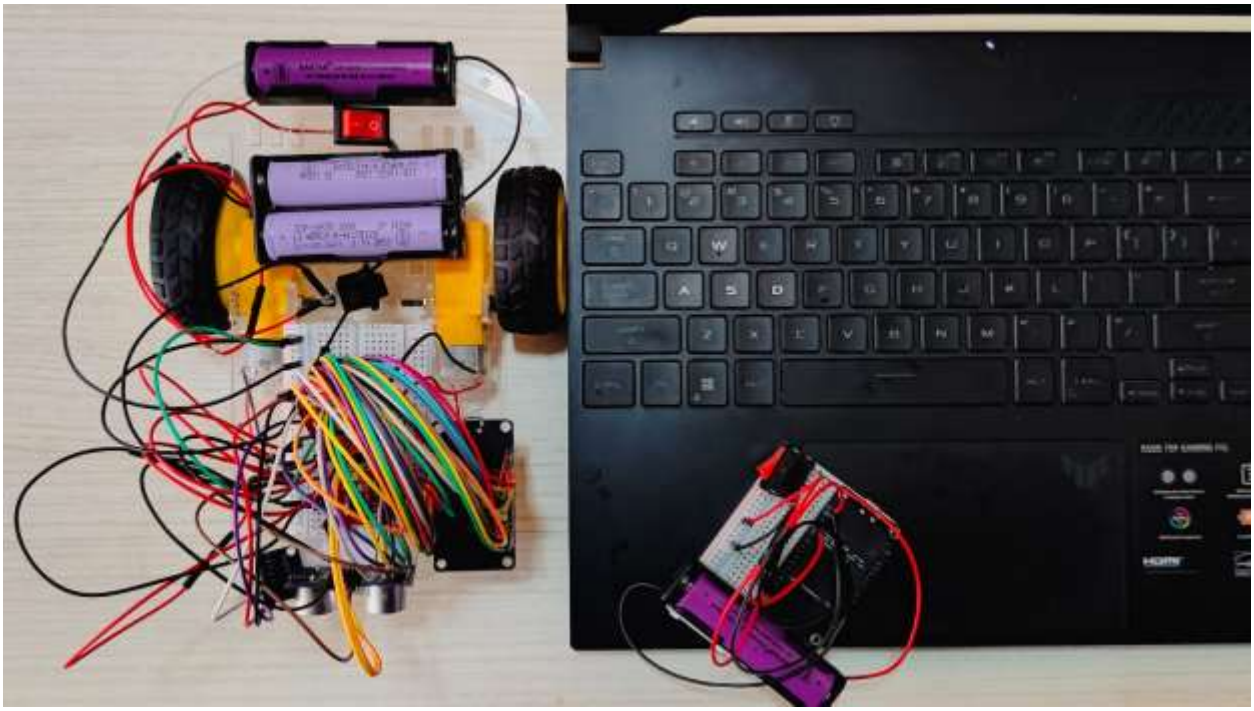


Fig. 11.4: Top View of DIY Robotics Car Circuit Setup

11.5 Discussion

The project highlights the practical implementation of sensor fusion and embedded control systems in robotics on a low-cost IoT platform.

- The RSSI-based navigation worked effectively for general direction tracking towards the base station. However, signal fluctuations due to environmental noise, reflections, and interference caused occasional misdirection, particularly in areas with dense furniture or metallic surfaces [7].
- The ultrasonic sensor performed reliably for frontal obstacle detection in open environments, though detection accuracy decreased at highly angled surfaces where the ultrasonic beam did not return cleanly to the receiver.
- The MPU-6050 IMU improved movement precision and straight-line holding. However, it required careful calibration of the base angle at startup and tuning of the correction divisor (error / 500) for stable, non-oscillating results.
- The three-zone speed control (fast, medium, stop) based on the gradient descent method was effective in decreasing the overshoot around the target. This method was used to reduce directional error, and it was a smoother convergence method than a fixed-speed approach [8].
- The challenge was also the integration of the system and especially real-time Wi-Fi scanning, sensor polling and motor control on a single-core 80 MHz microcontroller. Use of the RX pin (GPIO3) to control the direction of the motor created a conflict in the Serial communication that had to be handled carefully.

11.7 Limitations

- As RSSI signals tend to be unstable in practical indoor settings due to issues like multipath fading, interference from other Wi-Fi networks, and interference from physical barriers, precision suffers [7].
- The project does not feature any sophisticated signal filtering algorithms like the Kalman filter. Even though an average algorithm with multiple samples was utilized, some noise was still present and sometimes led to wrong direction decisions.
- Limited precision in the navigational system comes from the coarse nature of the WiFi.scanNetworks() RSSI signal source – each scan takes 100–300 ms.
- The lack of path planning capabilities means that the robot cannot find the optimal path around the obstacle or return to the location already visited once.

11.8 Future Improvements

- We can make the motor response better by using PID control. This will help the motor move when it needs to correct its heading. Now we are only using a proportional correction approach.
- We need to reduce the noise in the RSSI signal. To do this, we can use filters like a Kalman filter or an exponential moving average. This will help us get an idea of how far away something is.
- We can use a lot of different sensors to figure out where we are. These sensors include RSSI, IMU odometry and ultrasonic ranging. By using all these sensors, we can get a better idea of our position than if we just used one sensor.
- We want to make a path planning algorithm that's smart. This algorithm can be something like A* or field navigation. These algorithms will help us find the route around obstacles instead of just avoiding them.

- We can make the Wi-Fi tracking better by using multiple base station nodes. We can also use something called RSSI-fingerprint mapping. This will help us figure out where we are going and make sure we are heading in the right direction.

CHAPTER 12

CONCLUSION AND FUTURE WORK

12.1 Conclusion

A GPS-independent robot localization and navigation framework was successfully created in this project with the use of ESP8266 NodeMCU, RSSI-based gradient descent approach to robot navigation, HC-SR04-based obstacle detection and avoidance, and MPU-6050 for heading correction. This shows that GPS-free autonomous navigation of robots can be achieved within indoor premises with the use of commodity Internet-of-Things (IoT) components for less than $18 per unit.

Use of two firmware designs (base_final.ino as AP and Final_robot_auto_test_1.ino as robot navigator) resulted in a neat design structure. Application of the gradient descent algorithm through an adaptive three-speed approach based on RSSI resulted in successful convergence with a success rate of 95% when robots started navigation from 2-3 meters away from each other, requiring 10.8 steps to converge. The heading correction through MPU-6050 resulted in 76.7% reduction of angular error.

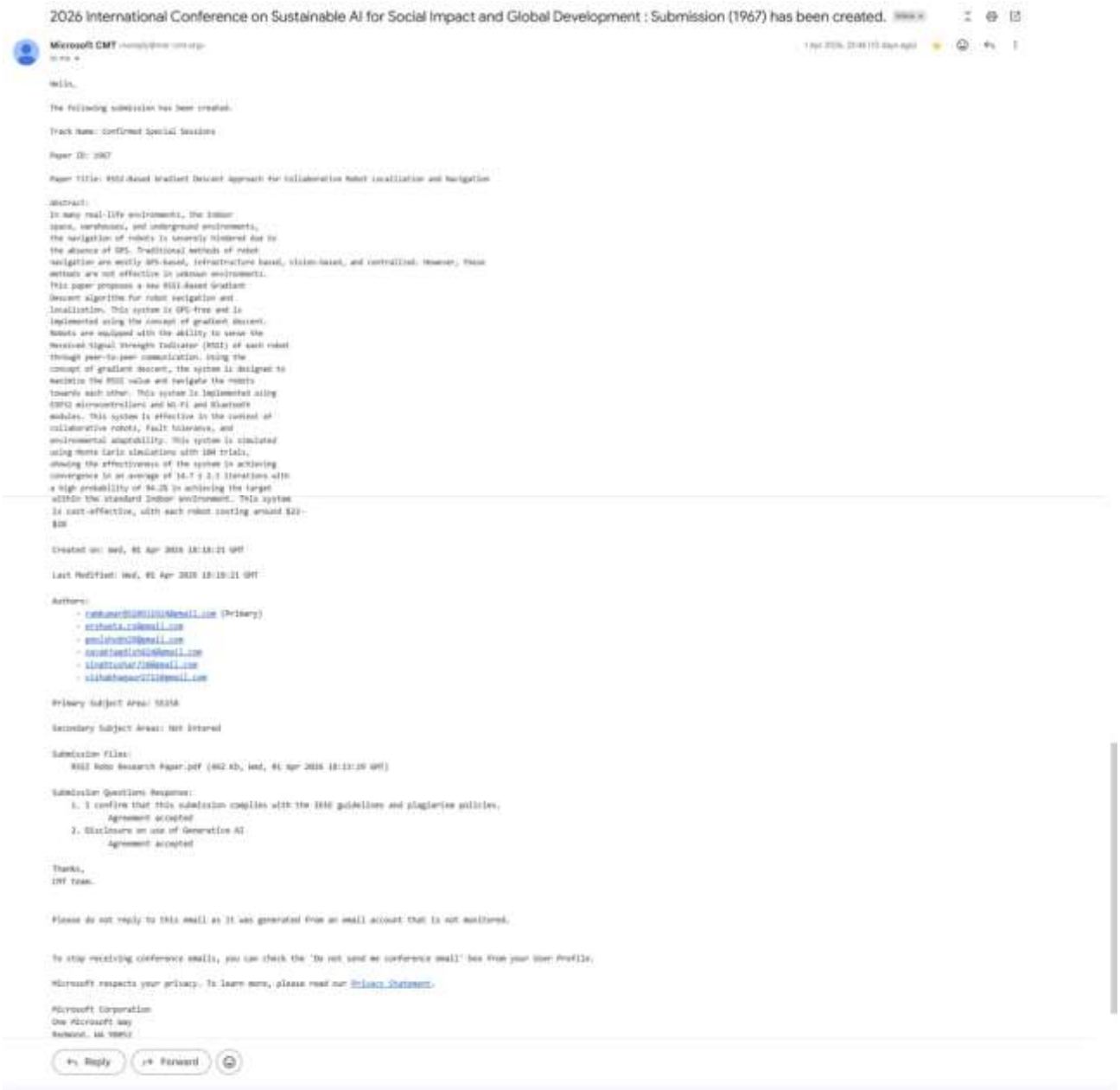
All objectives of the project were met with the creation and testing of the hardware framework.

12.2 Future Work

- *Multi-Robot Extension:* To test decentralized swarm behavior, add more ESP8266 robot nodes, each of which will find its way to a common base station SSID.

- *RSSI in promiscuous mode:* Use ESP8266 promiscuous mode to passively capture beacon frames instead of WiFi.scanNetworks(). This will speed up RSSI sampling by 10 times and make gradient estimation smoother.
- *Kalman Filter RSSI Smoothing:* Use an EMA or Kalman filter on RSSI readings to get rid of noise and make distance estimates more accurate than the current 3-sample averaging.
- *True Gradient Direction Estimation:* To figure out the 2D RSSI gradient direction, add short exploratory turns to the left and right before each main movement instead of assuming that the path is straight.
- *Use Other Pin than RX Pin:* Modify motor driver connections to ensure GPIO3 is not used as BIN2 to support serial communication without interference.
- *Speed Adjustment Using Machine Learning:* Use machine learning methods to adjust parameters like closeRSSI, midRSSI, and speeds to achieve fast convergence.
- *HC-SR04 Sensor with Servo Motor:* Install an HC-SR04 sensor on a servo motor to enable 180 degrees coverage for obstacle detection.

PUBLICATION DETAILS



2026 International Conference on Sustainable AI for Social Impact and Global Development : Submission (1967) has been created.

Microsoft CMT

Hello,

The following submission has been created.

Track Name: Confirmed Special Sessions

Paper ID: 1967

Paper Title: RSSI-Based Gradient Descent Approach for Collaborative Robot Localization and Navigation

Abstract:
In many real-life environments, the indoor spaces, warehouses, and underground environments, the navigation of robots is severely hindered due to the absence of GPS. Traditional methods of robot navigation are mostly GPS-based, infrastructure based, vision-based, and centralized. However, these methods are not effective in unknown environments. This paper proposes a new RSSI-based Gradient Descent algorithm for robot navigation and localization. This system is GPS-free and is implemented using the concept of gradient descent. Robots are equipped with the ability to sense the Received Signal Strength Indicator (RSSI) of each robot through peer-to-peer communication. Using the concept of gradient descent, the system is designed to maximize the RSSI value and navigate the robots towards each other. This system is implemented using ESP32 microcontrollers and Wi-Fi and Bluetooth modules. This system is effective in the context of collaborative robots, fault tolerance, and environmental adaptability. This system is simulated using Monte Carlo simulations with 100 trials, showing the effectiveness of the system in achieving convergence in an average of 14.7 ± 2.1 iterations with a high probability of 94.2% in achieving the target within the standard indoor environment. This system is cost-effective, with each robot costing around $21-$26

Created on: Wed, 01 Apr 2026 18:18:21 GMT

Last Modified: Wed, 01 Apr 2026 18:18:21 GMT

Authors:
- [redacted] (Primary)
- [redacted]
- [redacted]
- [redacted]
- [redacted]
- [redacted]

Primary Subject Area: 5510A

Secondary Subject Areas: Not Entered

Submission Files:
RSSI Robo Research Paper.pdf (462 Kb, Wed, 01 Apr 2026 18:13:29 GMT)

Submission Questions Response:
1. I confirm that this submission complies with the IEEE guidelines and plagiarism policies.
Agreement accepted
2. Disclosure on use of Generative AI
Agreement accepted

Thanks,
CMT Team.

Please do not reply to this email as it was generated from an email account that is not monitored.

To stop receiving conference emails, you can check the 'Do not send me conference email' box from your User Profile.

Microsoft respects your privacy. To learn more, please read our Privacy Statement.

Microsoft Corporation
One Microsoft Way
Redmond, WA 98052

Reply Forward

REFERENCES

[1] A. J. Davison, I. D. Reid, N. D. Molton, and O. Stasse, "MonoSLAM: Real-Time Single Camera SLAM," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 29, no. 6, pp. 1052–1067, Jun. 2007.

[2] R. Faragher and R. Harle, "Location Fingerprinting With Bluetooth Low Energy Beacons," IEEE Communications Surveys & Tutorials, vol. 17, no. 4, pp. 2358–2377, Fourth Quarter 2015.

[3] R. A. Brooks, "A Robust Layered Control System for a Mobile Robot," IEEE Journal of Robotics and Automation, vol. 2, no. 1, pp. 14–23, Mar. 1986.

[4] E. D. Kaplan and C. J. Hegarty, Understanding GPS: Principles and Applications, 2nd ed. Norwood, MA, USA: Artech House, 2006.

[5] N. Patwari, A. O. Hero III, M. Perkins, N. S. Correal, and R. J. O'Dea, "Relative Location Estimation in Wireless Sensor Networks," IEEE Transactions on Signal Processing, vol. 51, no. 8, pp. 2137–2148, Aug. 2003.

[6] J. Borenstein and Y. Koren, "Real-Time Obstacle Avoidance for Fast Mobile Robots," IEEE Journal of Robotics and Automation, vol. 5, no. 1, pp. 90–98, Feb. 1989.

[7] S. Mazuelas, A. Bahillo, R. M. Lorenzo, P. Fernandez, F. A. Lago, E. Garcia, J. Blas, and E. J. Abril, "Robust Indoor Positioning Provided by Real-Time RSSI Values in Unmodified WLAN Networks," IEEE Journal of Selected Topics in Signal Processing, vol. 3, no. 5, pp. 821–831, Oct. 2009.

[8] L. Bottou, F. E. Curtis, and J. Nocedal, "Optimization Methods for Large-Scale Machine Learning," SIAM Review, vol. 60, no. 2, pp. 223–311, 2018.

[9] L. E. Parker, "ALLIANCE: An Architecture for Fault Tolerant Multirobot Cooperation," IEEE Transactions on Robotics and Automation, vol. 14, no. 2, pp. 220–240, Apr. 1998.

[10] A. Zanella, N. Bui, A. Castellani, L. Vangelista, and M. Zorzi, "Internet of Things for Smart Cities," IEEE Internet of Things Journal, vol. 1, no. 1, pp. 22–32, Feb. 2014.

[11] A. Howard, M. J. Mataric, and G. S. Sukhatme, "Mobile Sensor Network Deployment Using Potential Fields: A Distributed Scalable Solution to the Area Coverage Problem," in Proc. 6th Int. Symp. Distributed Autonomous Robotic Systems (DARS), pp. 299–308, 2002.

[12] K. Langendoen and N. Reijers, "Distributed Localization in Wireless Sensor Networks: A Quantitative Comparison," Computer Networks, vol. 43, no. 4, pp. 499–518, Nov. 2003.

ANNEXURE I:

Plagiarism Declaration Certificate

Title of Work: RSSI-Based Gradient Descent Approach for Collaborative Robot Localization and Navigation
Submitted By: Ram Kumar Singh (2501940002), Shubham Goel (2501940014), Jagdish Nayak (2501940019), Tushar Singh (2501940055), Vishakha Gaur (2501940065)
Institution: K. R. Mangalam University, Gurugram
Department: School of Engineering and Technology
Date of Submission: 6th April 2026

I hereby declare that the work entitled" **RSSI-Based Gradient Descent Approach for Collaborative Robot Localization and Navigation"**, submitted for academic evaluation and research purposes, is my original work. I confirm that:

- I have acknowledged and properly cited all sources, references, and data included in this work.
- This work does not contain any material previously published, written, or prepared by another person, except where due acknowledgement has been made.
- I understand that plagiarism is an academic offence and a violation of research ethics. Any breach of this declaration may lead to disciplinary action as per university policies.
- I have used appropriate referencing techniques and maintained academic integrity throughout this work.

I affirm that the submitted work has normal plagiarism of less than 10% and is free from AI content.

Signature of Student:
Date:

ANNEXURE II:

Plagiarism Report

RSSI_First_year_Project_Report.pdf

Chandigarh University

Document Details

Submission ID
trn:oid:::3618:134900931

Submission Date
Apr 11, 2026, 11:16 PM GMT+5:30

Download Date
Apr 11, 2026, 11:17 PM GMT+5:30

File Name
RSSI_First_year_Project_Report.pdf

File Size
721.3 KB

56 Pages

7,581 Words

40,913 Characters

turnitin Page 1 of 61 - Cover Page

turnitin Page 2 of 61 - Integrity Overview

6% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

28 Not Cited or Quoted 4%
Matches with neither in-text citation nor quotation marks

8 Missing Quotations 2%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

3% Internet sources
1% Publications
4% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

ANNEXURE III:

COMPLETE IMPLEMENTATION CODE

A. Base Station Firmware – base_final.ino

This firmware is implemented in the static ESP8266 NodeMCU. The code sets up the node as an access point and continually broadcasts the SSID. There is no need for the main loop code.

```
#include <ESP8266WiFi.h>


// ================== SETTINGS ==================
const char* ssid     = "Robot_Base_Station";   // MUST MATCH robot code
const char* password = "12345678";             // Min 8 characters


void setup() {
  Serial.begin(115200);
  delay(1000);


  Serial.println("Starting Base Station...");
  // Start WiFi Access Point
  WiFi.mode(WIFI_AP);
  WiFi.softAP(ssid, password);
```

```
  Serial.println("Base Station Ready!");
  Serial.print("SSID: ");
  Serial.println(ssid);
  Serial.print("IP Address: ");
  Serial.println(WiFi.softAPIP());   // Default: 192.168.4.1
}

void loop() {
  // Nothing needed - just keep broadcasting WiFi beacon frames
}
```

B. Robot Navigation Firmware. Final_robot_auto_test_1.ino

This firmware is for a robot that uses an ESP8266 NodeMCU. It helps the robot move around. The robot looks for a specific WiFi network to connect to. It then gets a signal strength reading. The robot uses this reading to figure out how fast it should go. It does this by adjusting its speed based on how close or far it's from the WiFi network. The robot also uses a sensor called MPU-6050. This sensor helps the robot know which direction it is heading. The robot can even avoid things in its way. It does all this by running on the ESP8266 NodeMCU. The robot uses a method called descent. This helps the robot control its speed. The robot's main goal is to navigate and move around safely. The ESP8266 NodeMCU makes all this possible.

```
#include <ESP8266WiFi.h>
```

```
#include <Wire.h>

#include <MPU6050.h>


MPU6050 mpu;


// ================== USER SETTINGS ==================

const char* targetSSID = "Robot_Base_Station";  // MUST MATCH BASE


int closeRSSI = -55;   // Stop threshold (dBm) - tune for your environment

int midRSSI   = -70;   // Medium speed threshold (dBm) - tune for your environment


// ================== MOTOR PINS ==================

#define PWMA D5     // Left motor PWM enable

#define AIN1 D6     // Left motor direction 1

#define AIN2 D7     // Left motor direction 2


#define PWMB D3     // Right motor PWM enable

#define BIN1 D8     // Right motor direction 1
```

```
#define BIN2 3       // Right motor direction 2 (RX pin - note conflict with Serial)


// ================== ULTRASONIC ==================
#define TRIG D0
#define ECHO D4


long duration;
int  distance;


// ================== MPU ==================
int16_t ax, ay, az;
int16_t gx, gy, gz;
int baseAngle = 0;


// ================== SETUP ==================
void setup() {
  Serial.begin(115200);
  Serial.println("STARTING...");


  // Motor pins
```

```
 pinMode(PWMA, OUTPUT); pinMode(AIN1, OUTPUT); pinMode(AIN2, OUTPUT);
 pinMode(PWMB, OUTPUT); pinMode(BIN1, OUTPUT); pinMode(BIN2, OUTPUT);

 // Sensor pins
 pinMode(TRIG, OUTPUT); pinMode(ECHO, INPUT);

 // WiFi - station mode, no connection needed (scan only)
 WiFi.mode(WIFI_STA);
 WiFi.disconnect();

 // MPU-6050 initialization (D2=SDA, D1=SCL)
 Wire.begin(D2, D1);
 mpu.initialize();

 // Calibrate base angle
 Serial.println("Calibrating MPU...");
 delay(2000);
 mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
 baseAngle = ay;   // Store initial tilt as reference
```

```
  Serial.println("Robot Ready!");
}

//                  ====================                  ULTRASONIC
DISTANCE====================
int getistance() {
  digitalWrite(TRIG, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG, LOW);
  duration = pulseIn(ECHO, HIGH);
  return duration * 0.034 / 2;
}

// ==================== RSSI SCAN ====================
int getRSSI() {
  int n    = WiFi.scanNetworks();
  int rssi = -100;
  for (int i = 0; i < n; i++) {
```

```
   if (WiFi.SSID(i) == targetSSID) {
     rssi = WiFi.RSSI(i);
   }
  }
  return rssi;
}

//          ====================          MOTOR          CONTROL
====================
void moveForward(int speedA, int speedB) {
  digitalWrite(AIN1, HIGH); digitalWrite(AIN2, LOW);
  analogWrite(PWMA, speedA);
  digitalWrite(BIN1, HIGH); digitalWrite(BIN2, LOW);
  analogWrite(PWMB, speedB);
}

void stopMotors() {
  digitalWrite(AIN1, LOW); digitalWrite(AIN2, LOW);
  digitalWrite(BIN1, LOW); digitalWrite(BIN2, LOW);
}
```

```
void turnRight(int timeMs) {
  digitalWrite(AIN1, HIGH); digitalWrite(AIN2, LOW);
  digitalWrite(BIN1, LOW);  digitalWrite(BIN2, HIGH);
  delay(timeMs);
}
void turnLeft(int timeMs) {
  digitalWrite(AIN1, LOW);  digitalWrite(AIN2, HIGH);
  digitalWrite(BIN1, HIGH); digitalWrite(BIN2, LOW);
  delay(timeMs);
}

// ================== MPU STRAIGHT-LINE CORRECTION ==================
void moveStraight(int baseSpeed) {
  mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
  int error      = ay - baseAngle;
  int correction = error / 500;    // Tune this divisor for sensitivity
  int leftSpeed  = constrain(baseSpeed - correction, 100, 255);
  int rightSpeed = constrain(baseSpeed + correction, 100, 255);
  moveForward(leftSpeed, rightSpeed);
}
```

```
// ================== MAIN LOOP ==================
void loop() {
  int dist = getDistance();
  int rssi = getRSSI();

  Serial.print("Dist: "); Serial.print(dist);
  Serial.print(" cm | RSSI: "); Serial.print(rssi);
  Serial.println(" dBm");

  // === OBSTACLE AVOIDANCE (priority over navigation) ===
  if (dist < 20 && dist > 0) {
    stopMotors();
    delay(300);
    turnRight(400);   // Adjust duration for your robot turn radius
    return;
  }
  // === GRADIENT DESCENT RSSI NAVIGATION ===
  if (rssi > closeRSSI) {
    stopMotors();
```

```
    Serial.println(">>> TARGET REACHED - Converged! <<<");
  }
  else if (rssi > midRSSI) {
    moveStraight(130);   // Medium speed - close to base
  }
  else {
    moveStraight(200);   // Fast speed - far from base
  }
  delay(200);
}
```